Learning and Search in AlphaGo and AlphaGo Zero

Written assignment

Task 1 AlphaGo

DLMAIRIL01 Reinforcement Learning

IU International University of Applied Sciences

Master of Science Applied Artificial Intelligence

[Full name]

Student number [Student number]

Tutor [Tutor name]

Submission date [Date]

Contents

List of tables..... III
List of abbreviations..... III
1 Introduction..... 1
2 Foundations of learning and search..... 1
2.1 Go as a sequential decision problem..... 1
2.2 Why function approximation and selective search are complementary..... 2
3 How AlphaGo learns and plays..... 3
3.1 Neural networks and their training targets..... 3
3.2 Why reinforcement learning changes the objective..... 4
3.3 Position evaluation and the independence of training examples..... 5
3.4 How the networks support Monte Carlo Tree Search..... 6
4 How AlphaGo Zero changes the system..... 6
4.1 A shared network for policy and value..... 6
4.2 Search becomes a source of training targets..... 8
4.3 Neural evaluation without rollouts..... 8
5 Comparison and critical evaluation..... 10
5.1 What changes between the systems..... 10
5.2 What the empirical evidence establishes..... 11
5.3 Limits of self play and transfer..... 12
6 Conclusion..... 12
References..... 14

List of tables

Table 1 Functional comparison of AlphaGo Fan and AlphaGo Zero 10

List of abbreviations

AI Artificial intelligence

MCTS Monte Carlo Tree Search

RL Reinforcement learning

SL Supervised learning

UCT Upper Confidence bounds applied to Trees

1 Introduction

Go provides a useful setting for examining how learning and planning can support each other. Its rules determine the consequences of a legal move, yet choosing a strong move requires assessing alternatives whose consequences may emerge much later. A system can therefore possess an exact simulator without possessing an effective decision strategy. Monte Carlo Tree Search (MCTS) addresses this problem by combining selective exploration with estimates of future outcomes (Kocsis & Szepesvári, 2006, pp. 284–287).

This paper addresses the following question: How do neural networks and Monte Carlo Tree Search interact in AlphaGo, and how does AlphaGo Zero change this interaction through its architecture and learning procedure? The central argument is that AlphaGo combines learned guidance with local planning, whereas AlphaGo Zero also uses planning to generate the targets from which its network learns. Understanding this distinction requires separating the roles of a policy, a value estimate, and a search procedure.

The analysis is a focused literature review and conceptual comparison, not an experimental replication or systematic review. The course material supplied the initial orientation; the analysis itself relies on scientific publications. The primary studies by Silver et al. (2016, 2017) establish the systems and their reported evaluations; supporting methodological literature explains tree selection, policy gradients, residual learning, and the relationship between planning and generalisation. Sources are compared by component function, training target, information requirements, and evaluation conditions. Numerical results are reported from the studies, not presented as independently reproduced findings. For Silver et al. (2017), page locators refer to the numbered author accepted manuscript archived by UCL Discovery; this avoids confusing its pagination with that of the journal article.

The term AlphaGo primarily denotes the system described in the 2016 paper, subsequently called AlphaGo Fan. AlphaGo Lee is distinguished where the 2017 paper uses it as a benchmark. AlphaGo Zero is the self-play system described in 2017. Later generalisations and a complete survey of computer Go are outside the scope. The paper first defines the decision problem, then explains AlphaGo’s training and search, examines AlphaGo Zero, and evaluates what their comparison supports.

2 Foundations of learning and search

2.1 Go as a sequential decision problem

Go is a two-player game with alternating moves, deterministic transitions, and opposing objectives. A state must specify the information required by the rules, including whose turn it is and relevant repetition information. An action is a legal placement or pass. The transition model applies the

game rules; a terminal outcome supplies the learning signal. The original AlphaGo formulation treats the outcome as +1 for a win and −1 for a loss from the relevant player's perspective (Silver et al., 2016, Methods, "Problem setting").

This perspective matters when evaluating a sequence. A position favourable to the player who has just moved can be unfavourable to the opponent who moves next. Backups must therefore preserve a consistent player convention. A policy p(a|s) assigns probabilities to legal actions, whereas a value function estimates an outcome from a state. These objects answer different questions: which action should be considered, and how favourable is the resulting position? Neither is identical to the transition model that calculates the next board state.

The distinction also clarifies the role of model-based reasoning. AlphaGo does not need to learn the physical consequences of placing a stone; the rules already supply those consequences. Its learned uncertainty concerns decision quality and future play. Calling it a planning system therefore does not imply that a neural network learns the transition dynamics. The original system explicitly separates deterministic game transitions from the policy and outcome functions used to select actions (Silver et al., 2016, Methods, "Problem setting").

2.2 Why function approximation and selective search are complementary

Exhaustive enumeration is impractical when each position permits many moves and useful consequences lie many steps ahead. A selective search spends a limited budget on a subset of continuations. However, selectivity introduces a second problem: a promising action can receive too little attention if the procedure's current estimates are inaccurate. The allocation of computation must balance exploiting apparently strong actions and investigating less explored alternatives.

Kocsis and Szepesvári (2006, pp. 284–289) formulate this allocation through bandit-based Monte Carlo planning. Their UCT approach applies an upper-confidence selection principle within a tree. Its theoretical properties rely on specified assumptions and asymptotic analysis. Those results should not be interpreted as a proof that a finite neural-guided Go search always selects the optimal action. Their relevance here is the principle of allocating additional samples according to both estimated return and uncertainty.

A neural policy supplies reusable guidance before a particular search has accumulated much evidence. A value approximation provides a way to evaluate an intermediate position without always continuing to the end. Search, in turn, can revise the recommendation suggested by a network's first prediction. Learning amortises information across positions, while planning expends additional computation on the current decision. This interpretation explains why evaluating a network alone and evaluating the full search agent are different tests.

Two processes must be kept separate. Neural backpropagation computes gradients of a training objective and changes network parameters. A tree-search backup propagates a position evaluation along the visited path and changes tree statistics. A search can therefore improve its recommendation while the network weights remain fixed. Conversely, network training can improve future searches without preserving the individual tree used to produce a previous decision. Confusing these operations obscures how the components cooperate.

A simple outcome example clarifies the player convention. If Black ultimately wins, a position evaluated from Black's perspective has target +1, whereas the same continuation evaluated from White's perspective has target −1. A search that copied a favourable child value directly to the opponent's decision without changing perspective would prefer actions favourable to the wrong player. This is a consequence of the zero-sum formulation, rather than a detail that can be omitted when implementing backups. A value estimate near zero indicates uncertainty or approximately balanced expected outcomes under the represented play; it does not mean that the game necessarily ends in a draw.

The distinction between state and observation also matters. The visible placement of stones is insufficient to represent every rule-relevant aspect of a game if repeated positions are restricted. A practical representation therefore combines board information with history or other legality information. The simulator remains responsible for enforcing the rules. A compact neural input is a representation supplied to a function approximator, and should not automatically be assumed to establish a complete Markov state for every possible rule variant (Silver et al., 2017, pp. 22–23, 27).

3 How AlphaGo learns and plays

3.1 Neural networks and their training targets

The 2016 system uses separate convolutional policy and value networks. Its training pipeline starts with supervised prediction of expert moves, improves a policy through reinforcement learning, and trains a value estimator on outcomes generated by self-play. A separate fast rollout policy supports search evaluation. The rollout policy is a lightweight pattern-based model, not another deep convolutional network (Silver et al., 2016, pp. 484–486).

Convolution is appropriate because a board has spatial structure. Reusing filters across locations permits recurring configurations to share parameters. Successive layers can combine local relationships into more extensive patterns. This does not mean that a board position is an ordinary photograph: the input channels encode game state, and legal actions and outcomes are defined by rules. The architectural connection to computer vision is spatial representation, rather than an identity between the learning tasks.

Supervised policy learning uses expert actions as labels. If a training pair contains state s and expert move a*, an explanatory loss is:

$$L_{SL} = -\log p_{\sigma}(a^*|s) \quad (1)$$

Minimising this quantity increases the model's probability for the demonstrated move. It does not directly maximise the probability of eventually winning. A move can disagree with a human label and still be strong; a familiar-looking move can be strategically inferior in a particular continuation. Expert prediction is consequently a useful starting objective whose evaluation must remain distinct from playing strength.

The original study reports a 13-layer supervised policy trained on 30 million expert positions. Its input includes engineered features, such as liberties and capture information, alongside stone configuration. A softmax output provides move probabilities; the separate value network instead ends in a scalar outcome estimate. The reinforcement-learning policy starts from the supervised model and is improved through self-play. The value network is trained on outcomes from 30 million positions sampled from separate self-play games (Silver et al., 2016, pp. 485–486; Extended Data Table 2). These are separate learning problems even when the networks share similar convolutional building blocks.

3.2 Why reinforcement learning changes the objective

Policy-gradient reinforcement learning connects an action's probability to an outcome signal. Williams (1992) establishes the gradient-following basis of REINFORCE methods. For the episodic AlphaGo application, a simplified outcome-weighted update has the following form (Silver et al., 2016, p. 485):

$$\Delta\rho \propto z_t \nabla_{\rho} \log p_{\rho}(a_t|s_t) \quad (2)$$

Here ρ denotes policy parameters and z_t the final result expressed from the player's perspective at time t. A positive outcome reinforces sampled actions; a negative outcome shifts the update in the opposite direction. The expression explains the training principle, rather than reproducing every optimisation detail. A single result does not establish that every action in the game was good or bad. Useful improvement emerges statistically across experience.

For AlphaGo, this change of objective permits improvement beyond imitation. Nevertheless, a stronger policy in direct play need not be the best policy for allocating a limited search budget. A distribution concentrated on one action may play effectively on its own while offering an inadequate range of candidates to a search. This illustrates why the role of a component must determine its evaluation criterion.

The original training procedure also uses previous policy versions as opponents rather than relying exclusively on the latest policy playing an identical copy. This choice addresses the moving nature of the learning problem: when the policy changes, the distribution of visited positions and the

behaviour of the opponent change too. A policy that exploits a temporary opponent weakness may not be generally strong. Retaining earlier opponents exposes the learner to a broader set of responses, although it does not constitute a proof against all forms of overfitting (Silver et al., 2016, p. 485).

The practical distinction between imitation and reinforcement learning should therefore be assessed through the objective and the evaluation. Expert-move accuracy measures agreement with a dataset. Match performance measures success against specified opponents under specified conditions. The two can be related without being interchangeable. If an evaluation reports only prediction accuracy, it cannot establish that the resulting player wins more frequently. Conversely, winning a particular match does not establish that the player accurately reproduces expert behaviour. This distinction is central when interpreting the architectural comparisons discussed later.

3.3 Position evaluation and the independence of training examples

A value estimator predicts the expected game outcome under the behaviour represented in its training data. With signed outcomes, its scalar prediction is naturally interpreted on a scale from −1 to +1. It is an expectation, not an unqualified guarantee of victory. A convenient regression objective is:

$$L_{value} = \left(z - v_\theta(s)\right)^2 \tag{3}$$

The target z becomes known after a game ends. Although fitting this target is a regression operation, the data can arise from reinforcement learning through self-play. The origin of the experience and the numerical form of the fitting objective are different aspects of the method.

Training examples from one game are strongly dependent. Adjacent positions often differ only slightly and share the same outcome label. Treating them as independent evidence can allow memorisation of game-specific patterns and can make a dataset appear more informative than it is. The methodological lesson is to examine the unit of independence, not merely the number of positions. The original value-training experiment demonstrates this concern directly: selecting separate games for the sampled positions reduced the gap between training and test prediction error (Silver et al., 2016, p. 486).

This issue also limits interpretation of a value prediction. A network can make accurate average predictions on familiar self-play positions while being less reliable on unusual tactical states. Search can provide additional evidence, but it is itself guided by the same learned components. Combining methods is useful without making their errors automatically independent.

3.4 How the networks support Monte Carlo Tree Search

The original AlphaGo search uses the supervised policy as its move prior and combines a value-network estimate with a fast rollout result when evaluating a leaf. The reinforcement-learning policy contributes to generating value-training experience; it is not simply substituted for the supervised search prior. Silver et al. (2016, p. 486) report that the supervised prior performed better in the combined search and suggest diversity of candidate moves as an explanation.

The search repeats four conceptual stages. Selection follows actions favoured by a combination of estimated value and an exploration bonus. Expansion adds a newly considered position and its action priors. Evaluation obtains evidence about the leaf's outcome. Backup incorporates that evidence into the counts and values of the visited edges. AlphaGo implements these stages with asynchronous neural evaluation and rollout computation (Silver et al., 2016, p. 486; Methods, "Search algorithm").

An explanatory expression for the combined leaf evaluation is:

$$V(s_L) = (1 - \lambda) v_\theta(s_L) + \lambda z_L \quad (4)$$

The value prediction supplies a learned estimate; z_L is the outcome of a rollout completed under the fast policy. The parameter λ controls their relative contribution. This is a combination of evaluators with different computational and statistical characteristics. It is not a combination of an uncertain estimate with a guaranteed optimal answer: a rollout is also dependent on the policy used to complete the game.

During repeated simulations, an edge accumulates evidence about continuations beneath it. A high prior gives an action early attention, but subsequent search evidence can alter its importance. Once the search budget is exhausted, the original system selects the root action with the highest visit count (Silver et al., 2016, p. 486). Thus the network's most probable move need not become the move played.

The result is a division of labour. The policy guides the breadth of exploration, the value estimator reduces the need for long continuations, and the tree organises additional computation around the present decision. More accurate predictions can improve the use of a fixed search budget, but slower networks can also reduce the number of evaluations affordable within that budget. Component quality must therefore be assessed together with computational cost.

4 How AlphaGo Zero changes the system

4.1 A shared network for policy and value

AlphaGo Zero replaces separate policy and value models with one residual network containing a shared representation and two output heads. It starts from random parameters and learns from

self-play without expert game labels. It also removes rollout evaluation. Its input has 17 binary planes: eight successive board positions represented separately for each player and one colour-to-play plane. The policy head produces 362 action scores for the 361 intersections and pass; the value head ends in a tanh output on [−1, +1] (Silver et al., 2017, pp. 3–6, 27–29).

A shared representation permits policy and value learning to influence the same internal features. This can reduce duplication, but sharing is a design choice whose effect requires evidence: two objectives do not necessarily benefit from identical features in every domain. The relevant claim is that sharing worked in this system, rather than that a shared network must always outperform separate models.

Residual learning provides another important distinction. He et al. (2016, pp. 770–773) introduce blocks that learn a residual transformation with respect to their input. In a simplified block, the output is:

$$y = x + F(x) \tag{5}$$

The shortcut provides a direct path for information and gradients, making deep networks easier to optimise in their experiments. A residual block is not synonymous with a single convolutional layer. Consequently, comparing a layer count in the original policy network with a residual-block count in Zero without explaining the units would misrepresent architectural depth.

The shared residual tower in Zero begins with a convolutional block and is followed by residual blocks. Each residual block contains two convolutions with batch normalisation, rectified activations, and a shortcut connection. The two heads then transform the shared representation for different outputs: the policy head calculates action scores, while the value head reduces the spatial representation to an outcome estimate. The reported smaller and larger configurations use different depths; their training durations and performance results therefore belong to separate experimental configurations (Silver et al., 2017, pp. 27–29).

This architecture provides a concrete connection to computer vision without making visual recognition the objective of the Go task. Spatial filters learn recurring local structure, and residual connections support optimisation of the deep representation. Yet strategic value also depends on interactions across the board. Local pattern recognition alone cannot replace the decision procedure. The rationale for combining convolution with search is that reusable pattern information can guide the investigation of consequences that depend on the current position. The architectural benefit and the planning benefit should be evaluated separately before assessing their joint effect.

4.2 Search becomes a source of training targets

Zero's network outputs a policy distribution p and a scalar value v. Search uses these predictions to obtain a refined move distribution π. Completed self-play games supply signed outcomes z. Training then fits both outputs to these targets:

$$(p,\ v) = f_\theta(s)$$

$$L = (z - v)^2 - \Sigma_a\ \pi_a\ \log\ p_a + c\|\theta\|^2 \qquad (6)$$

The first term penalises outcome-prediction error. The second is cross-entropy against search probabilities, and the last discourages excessively large parameters. The distinctive training target is the distribution produced by search, rather than an expert's recorded move. This objective is specified by Silver et al. (2017, p. 6, Equation 1).

The interaction can be understood through policy improvement and evaluation. Search spends computation comparing continuations and provides a stronger local recommendation than the network's initial output. Training generalises these recommendations across states. Self-play outcomes supply evidence about the resulting behaviour. Repeating these operations allows the network and its search procedure to improve together, subject to approximation error and the quality of generated experience.

Anthony et al. (2017, pp. 1–3) offer a closely related conceptual framework in Expert Iteration. They distinguish a planning expert from an apprentice that generalises the expert's decisions, and use the apprentice to improve subsequent planning. Their experiments concern Hex, so they do not independently reproduce AlphaGo Zero. Their relevance is methodological: they explain why learning from planning can be different from requiring a policy network to discover good behaviour without that planning support.

This perspective also resolves an apparent contradiction. Zero uses supervised-style losses, yet its overall learning process is reinforcement learning. Cross-entropy specifies how the network fits a target; it does not establish where that target originated. In Zero, the targets arise from the agent's own search and game outcomes. Describing the system as human-supervised because it uses cross-entropy would therefore confuse optimisation with supervision source.

4.3 Neural evaluation without rollouts

Removing rollouts does not remove tree search. A search simulation can stop at a newly evaluated leaf instead of playing an additional game to completion. The value output supplies the backed-up evaluation, while the policy output supplies priors for subsequent exploration. MCTS therefore remains a planning procedure even when it no longer relies on the original fast rollout evaluator.

For Zero, selection uses a prior-weighted exploration term of the following form:

$$U(s,a) = c_{puct}\ P(s,a)\ \frac{\sqrt{\Sigma_b\ N(s,b)}}{1 + N(s,a)} \tag{7}$$

Here P is the network prior, N an edge visit count, and c_{puct} the exploration coefficient. Selection maximises Q + U. The numerator links exploration to activity at the parent, while the denominator reduces the bonus for an already visited action. This is Zero's specified form; it should not be presented as an identical transcription of every earlier AlphaGo search implementation (Silver et al., 2017, pp. 25–27).

An illustrative consequence follows directly from the expression. If two actions have equal estimated values and visit counts, the larger prior receives the larger exploration bonus. If one action is repeatedly selected while the other remains less visited, their relative bonuses change. The procedure can therefore revisit alternatives without treating every legal action equally at every step. It still depends on finite computation and imperfect values.

The resulting visit distribution is temperature-controlled: $\pi(a|s)$ is proportional to $N(s,a)^{1/\tau}$. Higher temperature spreads selection more widely, while a temperature approaching zero concentrates selection on the most visited action. Root noise during self-play supplies additional exploration. Candidate networks are evaluated before replacing the data-generating incumbent (Silver et al., 2017, pp. 23–25). These mechanisms matter because a self-generated curriculum can otherwise repeatedly reinforce a narrow set of experiences.

The stored learning example can be written as a tuple consisting of a state, its search distribution, and the eventual outcome. This makes two training signals available from one game: the search distribution describes the allocation of attention among candidate moves, while the outcome indicates success from the player's perspective. Fitting a distribution retains information about alternatives that would be lost if the target were reduced to a single selected action. The outcome target, however, remains a sample from self-play rather than a proof of the position's value under perfect play (Silver et al., 2017, pp. 5–6).

Exploration must operate at the scale of complete games as well as individual searches. If the opening policy repeatedly generates the same positions, accurate search in those positions does not by itself ensure that other useful regions are visited. The original Zero procedure therefore samples moves more broadly during the opening and injects noise into root priors. This encourages variation while allowing search evidence to outweigh an unpromising initial suggestion. The evaluator admits a new checkpoint only after a sufficient measured advantage against the incumbent; this is a mechanism for controlling updates to the self-play generator, not an assertion that every gradient step improves playing strength (Silver et al., 2017, pp. 23–25).

5 Comparison and critical evaluation

5.1 What changes between the systems

Table 1 summarises the functional comparison. The most consequential difference is where the policy-training targets originate. Expert demonstrations help establish the original policy, whereas search-guided self-play generates Zero's targets from the beginning. The change in architecture supports this procedure, but architecture and learning algorithm must remain distinct explanatory factors.

Table 1

Functional comparison of AlphaGo Fan and AlphaGo Zero

Dimension	AlphaGo Fan	AlphaGo Zero
Initial experience	Expert demonstrations followed by self-play	Self-play from random network parameters
Deep models	Separate policy and value networks	Shared residual network with policy and value heads
State features	Board information and engineered tactical features	Board history and colour to play
Policy improvement	Policy-gradient learning after supervised initialisation	Fitting search-generated move distributions
Search priors	Supervised policy network	Policy head of the joint network
Leaf evaluation	Value network combined with a fast rollout	Value head without rollout evaluation
Learning from search	Original value data generated by RL-policy self-play	Search guides self-play and supplies policy targets

Note. Author synthesis based on Silver et al. (2016, pp. 484–486) and Silver et al. (2017, pp. 3–6, 21–22).

Removing expert games eliminates one dependency, but it does not remove human design. Rules, legal-action generation, the board representation, network architecture, reward definition, and optimisation settings remain specified by researchers. "Without human knowledge" should therefore be interpreted in the qualified sense examined in the paper's domain-knowledge discussion, rather than as absence of every prior assumption (Silver et al., 2017, pp. 22–23).

The comparison also separates representation learning from planning. A convolutional architecture does not by itself establish a reinforcement-learning method, and a network with policy and value outputs is not by itself the complete agent. The decision procedure includes how search uses those outputs. Reproducing only the architecture would leave the central learning-and-search interaction unspecified.

5.2 What the empirical evidence establishes

The original paper reports a 5–0 formal match victory over Fan Hui. The later paper reports a 100–0 result against AlphaGo Lee after approximately three days of training and 4.9 million self-play games. A separate 40-day run generated 29 million games and defeated AlphaGo Master 89–11 (Silver et al., 2016, p. 488; Silver et al., 2017, pp. 6–8, 10–12). These outcomes concern different opponents and configurations. They should not be merged into a single benchmark or attributed to one unchanged AlphaGo version.

Match victories establish strong performance under the reported playing conditions. They do not isolate the causal contribution of each change. Between the systems, training targets, input representation, architecture, search evaluation, and computational arrangements differ. Attributing the full improvement exclusively to removing human demonstrations would therefore exceed the evidence of the headline match result.

Component comparisons provide more targeted evidence. The original paper compares rollout-only, value-only, and combined evaluation. The Zero paper examines architectural alternatives, including shared and separate outputs (Silver et al., 2016, pp. 487–488, Figure 4; Silver et al., 2017, pp. 8–9, Figure 4). Such analyses are more informative about mechanisms than a single final score. Even so, their conclusions are conditional on training schedules, search budgets, and the other components held in place.

Hardware comparisons require similar care. A smaller playing configuration does not establish a proportionally smaller total training cost. Self-play generation, network optimisation, checkpoint evaluation, and competitive inference consume different resources. A rigorous efficiency claim would specify the phase, hardware, elapsed time, and amount of computation. Without a common accounting basis, the defensible conclusion is improved performance with a different system design, rather than a precise universal efficiency multiplier.

An additional evaluation issue concerns the distribution on which an architectural comparison is performed. Training different architectures on a fixed dataset helps separate representation and prediction effects from differences in experience generation. However, the dataset itself was generated by an existing agent. It therefore represents the states encountered by that generator, rather than an exhaustive sample of Go. An architecture that performs well on this distribution still needs to be evaluated when its own decisions determine subsequent experience. This is why prediction tests and full-agent matches provide complementary evidence (Silver et al., 2017, pp. 8–9, 29–30).

The reported architectural results also caution against using one metric as a proxy for the whole system. Sharing the representation can improve outcome estimation and playing strength even when agreement with human moves does not improve. The relevant design question is whether the component helps the complete agent meet its objective under the available resources. On this

interpretation, architectural simplicity is valuable when it supports useful computation and stable learning; the number of networks alone is not an adequate measure of complexity or performance.

5.3 Limits of self play and transfer

Go provides unusually favourable conditions for autonomous learning: an exact simulator, unambiguous legal actions, repeatable interaction, and a score that identifies success. Industrial decision problems may have uncertain dynamics, delayed measurements, changing operating conditions, and costly failures. Consequently, the availability of self-play in Go does not establish that replacing measured data with simulated experience is appropriate in a physical system.

The transferable idea is to combine learned generalisation with focused planning where a sufficiently reliable model exists. An industrial application would need to justify its simulator, specify operational constraints, and test the resulting decisions against conditions beyond those used for development. This is an implication of the comparison, not an empirical result demonstrated by the Go studies. The underlying requirement is already visible in Monte Carlo planning: the procedure assumes access to a generative model of the decision problem (Kocsis & Szepesvári, 2006, pp. 282–284).

Self-play performance also does not demonstrate general intelligence, human-like understanding, or mathematical optimality. It shows the effectiveness of a particular learning-and-search procedure within a specified environment. Keeping this boundary explicit improves the argument: the systems are important because their mechanisms are identifiable and testable, without needing broader claims unsupported by their evaluations.

6 Conclusion

AlphaGo addresses difficult sequential decisions by assigning complementary functions to learned models and search. Its policy guides exploration, its value estimator assesses positions, and MCTS gathers additional evidence before selecting a move. The original pipeline combines expert supervision with reinforcement learning and retains a separate rollout evaluator. Explaining these roles separately is essential because the strongest stand-alone component need not be the most useful component inside the search.

AlphaGo Zero changes both representation and the source of improvement. A shared residual network provides policy and value predictions, rollout evaluation is removed, and search-generated move distributions become policy-training targets. The answer to the research question is therefore that the systems combine learning with planning at decision time, while Zero more directly closes the loop between planning, experience generation, and network training. Search helps determine not only which move to play but also what the policy should learn.

The reported results support the effectiveness of this combination in Go. They do not establish that expert data is universally harmful, that all gains arise from one architectural change, or that the approach transfers unchanged to physical systems. The comparison is limited by its reliance on published experiments and by differences between system configurations. A useful next investigation would control architecture, search budget, and training computation while varying the source of policy targets. For applied reinforcement learning, the corresponding question is whether planning generates reliable improvement targets under the model uncertainty and resource constraints of the intended environment.

References

Anthony, T., Tian, Z., & Barber, D. (2017). Thinking fast and slow with deep learning and tree search. In I. Guyon, U. von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. V. N. Vishwanathan, & R. Garnett (Eds.), *Advances in neural information processing systems* (Vol. 30). Curran Associates. https://proceedings.neurips.cc/paper/2017/hash/d8e1344e27a5b08cdfd5d027d9b8d6de-Abstract.html

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770–778). IEEE. https://doi.org/10.1109/CVPR.2016.90

Kocsis, L., & Szepesvári, C. (2006). Bandit based Monte-Carlo planning. In J. Fürnkranz, T. Scheffer, & M. Spiliopoulou (Eds.), *Machine learning ECML 2006* (pp. 282–293). Springer. https://doi.org/10.1007/11871842_29

Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature, 529*(7587), 484–489. https://doi.org/10.1038/nature16961

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., & Hassabis, D. (2017). *Mastering the game of Go without human knowledge* [Accepted manuscript]. UCL Discovery. https://discovery.ucl.ac.uk/id/eprint/10045895/1/agz_unformatted_nature.pdf

Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning, 8*(3–4), 229–256. https://doi.org/10.1007/BF00992696